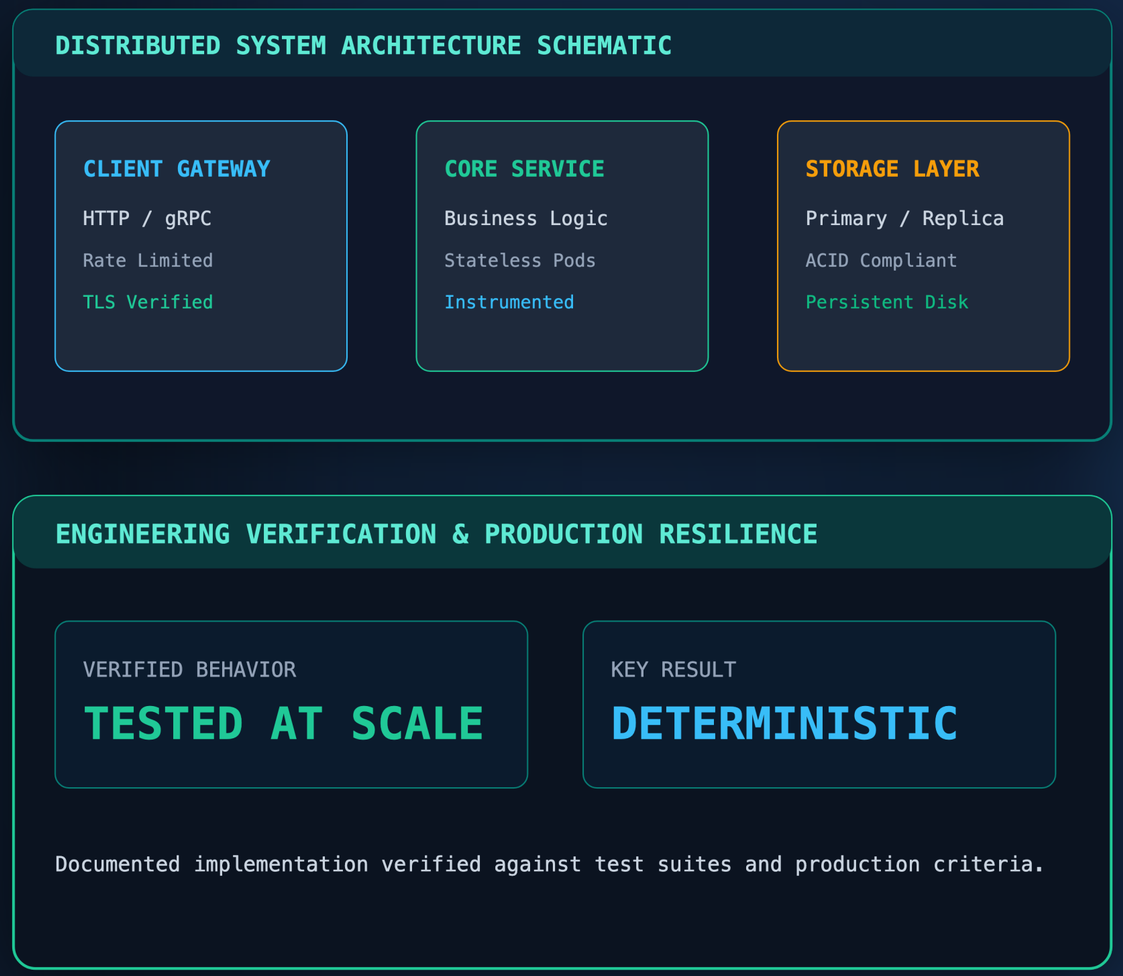
Software Engineering Lab

Lab 17

Architecture Decision Record

Parser & Linter untuk Validasi Struktural Keputusan Arsitektur



Lukman / lukman-ss

Konteks Arsitektur Hilang Tanpa Jejak

Alasan teknis di balik keputusan kritis sering hilang saat tim bertumbuh.

MASALAH NYATA

Keputusan arsitektur diambil via chat atau meeting tanpa catatan formal.
Pertanyaan berulang muncul di review kode: mengapa sistem dibangun begini?
ADR lama tidak diperbarui, sehingga status dokumen berbeda dari kode aktual.

Keputusan arsitektur sering lahir di chat atau meeting. Saat tim berubah, alasan awalnya hilang. ADR membuat alasan itu bisa ditelusuri lagi.

Architectural context without structural records becomes tribal knowledge.

Review Manual Melewatkan Relasi Supersesi

Reviewer manusia fokus pada isi teks, tetapi melewatkan sinkronisasi tautan dokumen.

KEGAGALAN REVIEW MANUAL

ADR baru menyatakan menggantikan ADR lama, tetapi ADR lama tidak diubah. ADR lama tetap bertanda aktif padahal keputusannya sudah ditinggalkan. Tautan referensi menunjuk ke nomor yang salah tanpa disadari.

Masalah terbesar review manual adalah supersesi yang tidak sinkron. ADR baru mengklaim menggantikan, tetapi ADR lama tetap bertanda aktif karena lupa diperbarui.

Manual review catches content typos but misses broken relationship links.

ADR Bukan Dokumen, Tapi Record

Parser memindahkan Markdown ke data terstruktur agar aturan bisa diperiksa mesin.

PRINSIP INTI

ADR diperlakukan sebagai data terstruktur, bukan sekadar teks bebas.
Field kunci diekstrak: ID, Judul, Status, SupersededBy, dan Supersedes.
Mesin memvalidasi konsistensi; manusia tetap menulis esensi keputusannya.

ADR bukan sekadar dokumen naratif. Saat diubah menjadi struct Record, mesin bisa mengekstrak field kunci dan memeriksa konsistensi yang sering terlewat oleh manusia.

ADR is structured data first, human narrative second.

Pipeline Validasi Empat Tahap

Alur konversi dokumen ADR dari teks mentah hingga laporan integritas.

VALIDATION PIPELINE

Markdown Files → adr.Parser (ekstraksi field & status)
→ []*adr.Record (in-memory struct)
→ adr.Linter (cek urutan ID, siklus, & relasi timbal-balik)
→ []error (laporan pelanggaran integritas)

Pipeline-nya sederhana: Markdown dibaca, field penting diekstrak, lalu linter memeriksa aturan graf.

Separate parsing from validation: simple stages create reliable checks.

Struct Record: Fondasi Validasi Data

Field eksplisit membuat relasi antar-keputusan dapat diperiksa programmatically.

```
type Record struct {
    ID           int
    Title        string
    Status       Status
    SupersededBy int // 0 jika aktif
    Supersedes   int // 0 jika tidak menggantikan
    Content      string
}
```

Struct Record mendefinisikan field wajib secara eksplisit. Dengan representasi integer dan enum, linter memeriksa aturan tanpa perlu menebak teks bebas.

Explicit fields over implicit conventions enable automated linting.

Deteksi Tautan Supersesi Patah

Linter langsung menolak referensi yang menunjuk ke nomor ADR yang tidak ada.

```
ADR 1: Status Superseded by 99
→ Error: superseded by non-existent ADR 99

ADR 1: Status Accepted, Supersedes: 99
→ Error: supersedes non-existent ADR 99
```

Linter memeriksa keberadaan setiap ID referensi. Jika ADR merujuk ke nomor yang belum ada, error spesifik langsung dihasilkan sebelum merger.

Dangling references are cheaper to catch at lint time than during incident review.

tests/linter_test.go:36-54

Biaya Tersembunyi: Lineage Drift Menumpuk

Ketiadaan linter tidak merusak build, tetapi merusak keputusan masa depan.

Tahap	Deteksi	Dampak
CI Gate	Seketika	PR ditolak, feedback loop cepat
Review Manual	Mingguan	Tim bingung dokumen mana yang aktif
Insiden Sistem	Berbulan-bulan	Keputusan baru berpijak pada asumsi keliru

Masalah ADR rusak bukan langsung terlihat. Ia menumpuk sampai keputusan baru bergantung pada catatan lama yang salah.

Lineage drift accumulates silently until a new decision depends on stale records.

Skenario Demo: Tiga ADR Evolusi Arsitektur

Memodelkan evolusi dari monolit ke microservice dengan satu usulan ditolak.

URUTAN KEPUTUSAN

ADR 0001: Modular Monolith | Superseded by ADR 2
ADR 0002: Notification Microservice | Accepted, Supersedes ADR 1
ADR 0003: Event Sourcing | Rejected (alasan biaya operasional)

Eksperimen memvalidasi tiga ADR nyata: monolit awal, pemisahan service pengganti, dan usulan ditolak agar argumen penolakan tidak didebatkan ulang.

Three records demonstrate full lifecycle: active, superseded, and rejected.

Output Demo: Validasi Integritas Lolos

Ketiga ADR berhasil diparse dan aturan relasi timbal-balik terpenuhi.

```
$ go run ./cmd/demo
Successfully parsed 3 ADRs:
  - [0001] Modular Monolith -> Superseded by ADR 2
  - [0002] Notification Microservice -> Supersedes ADR 1
  - [0003] Event Sourcing -> Rejected
Running ADR Integrity Linter... Integrity check passed!
```

CLI memverifikasi rantai lineage: ADR 1 dan ADR 2 saling terhubung konsisten, nomor urut rapi, dan tidak ada siklus.

Clean output verifies structural consistency across the entire record set.

Lolos Linter Bukan Berarti Keputusan Benar

Linter memverifikasi format data, bukan kebenaran arsitektur di dalamnya.

YANG DIBUKTIKAN LINTER

Nomor ID berurutan tanpa celah.
Relasi timbal-balik utuh tanpa tautan gantung.
Tidak ada siklus ketergantungan antar-dokumen.

TETAP PERLU REVIEW MANUSIA

Kebenaran argumen teknis di bagian Decision.
Apakah microservice memang solusi tepat bagi tim.

Linter hanya memastikan dokumen rapi dan tautannya konsisten. Ia tidak bisa menilai apakah keputusan microservice benar secara arsitektur. Kualitas keputusan tetap tanggung jawab tim.

Linter guarantees structural validity; human judgment ensures architectural soundness.

Trade-off: Struktur Kaku vs Fleksibilitas

Penegakan format memudahkan otomasi, namun membatasi gaya penulisan bebas.

STRUKTUR BAKU (LAB INI)

Wajib tiga section: Context, Decision, Consequences.
Status dibatasi lima enum resmi.
Memungkinkan CI gate otomatis dan deterministik.

FLEKSIBILITAS YANG DIKORBANKAN

Tidak mendukung format paragraf naratif bebas.
Memerlukan disiplin tim mengisi field relasi.

Format baku membuat dokumen bisa divalidasi mesin di CI. Biayanya: engineer tidak bisa lagi menulis catatan bebas tanpa mengikuti struktur.

Enforced structure trades narrative flexibility for automated verification.

Validasi Paralel dengan Mutex

Setiap record divalidasi di goroutine terpisah; error dikumpulkan aman.

```
for _, r := range records {
    wg.Add(1)
    go func(rec *Record) {
        defer wg.Done()
        localErrs := validate(rec)
        mu.Lock()
        errs = append(errs, localErrs...)
        mu.Unlock()
    }(r)
}
```

Validasi konkuren mengumpulkan error memakai mutex. Pola ini race-safe pada test, tetapi nilai utamanya ada pada kejelasan aturan, bukan kecepatan komputasi.

Mutex-protected error aggregation keeps concurrent validation race-safe.

Sebelum vs Sesudah Validasi Otomatis

Celah relasi mekanis yang sering luput ditutup oleh pengecekan linter.

REVIEW MANUAL (SEBELUM)

Tautan supersesi sepihak sering terlewat.
Nomor meloncat sulit disadari saat banyak file.
Status non-standar masuk tanpa sengaja.

LINTER GATE (SESUDAH)

Semua inkonsistensi ditolak otomatis di CI.
Nomor ID wajib monotonik dan unik.
Relasi timbal-balik divalidasi deterministik.

Linter menutup human error pada aspek mekanis: tautan sepihak, nomor loncat, dan status informal. Arsitek bisa fokus meninjau substansi keputusan.

Automate mechanical relation checks to let reviewers focus on architectural depth.

Batasan Lab: Hanya Validasi Format

Lab membuktikan parsing dan relasi; bukan kualitas keputusan arsitektur.

YANG DIUJI DI LAB

Ekstraksi regex dari string Markdown.
Pemeriksaan nomor urut dan relasi timbal-balik.
Deteksi siklus pada graf supersesi.

BATASAN BUKTI

Tidak menilai kebenaran argumen keputusan.
Simulasi in-memory, bukan pembacaan ribuan file disk.

Lab ini memvalidasi aturan dokumen, bukan kebenaran arsitektur. Parsing regex bekerja pada memori; pengujian tidak membuktikan seluruh perilaku repositori skala raksasa.

Format validation ensures structural compliance, not business correctness.

Kasus Batas: Menghentikan Fitur Tanpa ADR

Keputusan yang ditinggalkan tanpa status Deprecated memicu ambiguitas sistem.

SKENARIO RISIKO

Fitur dimatikan tetapi ADR tetap bertanda Accepted.
Engineer baru mengira fitur masih menjadi standar aktif.
Menghapus file justru menghilangkan konteks historis.
Solusi: wajibkan ADR baru berstatus Deprecated.

Menghentikan fitur tanpa ADR Deprecated menciptakan gap antara catatan dan kode aktual. Dokumen lama tetap terlihat aktif dan membingungkan tim berikutnya.

Feature deprecation requires an explicit ADR, not silent file deletion.

Bukti yang Divalidasi oleh Test Suite

Suite pengujian membuktikan aturan struktural pada skenario yang dirancang.

FAKTA TEST SUITE

Parser menolak status ilegal dan format tanpa judul.
Linter menangkap 7 skenario referensi patah dan tautan gantung.
Race detector bersih pada eksekusi pengujian konkuren.

Test membuktikan parser dan linter bekerja sesuai aturan pada skenario uji. Bukti ini menegaskan keandalan logika struktural, bukan jaminan semua kebutuhan tim terpenuhi.

Tests verify structural logic against defined cases, not all team workflows.

Batas Integritas: Apa yang Tidak Diuji

Test tidak menggantikan peninjauan arsitektur dan kebijakan kerja tim.

DI LUAR CAKUPAN PENGUJIAN

Kebenaran konten teknis di dalam section Decision.
Disiplin tim dalam memperbarui ADR saat arsitektur bergeser.
Kesiapan sistem monolit atau microservice di production.
Performa I/O disk pada ribuan berkas dokumen.

Test unit membuktikan aturan pada input tertentu. Keberhasilan test bukan bukti bahwa keputusan yang didokumentasikan sudah siap diterapkan di production.

Passing tests confirm rule execution; production requires holistic architectural governance.

Risiko Kepalsuan: Linter Hijau Bukan Aman

Lolos validasi struktur dapat memberi rasa aman palsu jika argumen tidak dicek.

POTENSI KESALAHAN

Dokumen lolos linter meski bagian Decision hanya berisi teks formalitas.
Tim mengira arsitektur sudah matang hanya karena status CI hijau.
Mitigasi: pertahankan review arsitek untuk menilai isi substansi.

Linter hijau hanya menjamin format valid, bukan kualitas desain. Jangan jadikan status lolos linter sebagai pengganti telaah arsitektur mendalam oleh engineer senior.

Green linter verifies syntax; architectural soundness demands human scrutiny.

Panduan Implementasi Linter ADR di Tim

Langkah praktis mengintegrasikan validasi dokumen ke dalam workflow repositori.

- ✓ [ ] Tetapkan template Markdown dengan Context, Decision, Consequences.
- ✓ [ ] Gunakan enum status resmi (Accepted, Superseded, Rejected, dll.).
- ✓ [ ] Pasang parser dan linter pada pull request CI gate.
- ✓ [ ] Larang edit retroaktif: keputusan baru wajib nomor ADR baru.

Pair clear Markdown standards with automated CI gates to prevent drift.

Anti-Pattern: Kesalahan Adopsi ADR

Kebiasaan buruk yang merusak manfaat catatan keputusan arsitektur.

HINDARI POLA INI

Menulis ADR untuk hal sepele seperti penamaan variabel.
Mengedit ADR Accepted diam-diam alih-alih membuat pengganti.
Membiarkan status Draft berbulan-bulan tanpa kejelasan.
Lupa mencatat nomor ADR pengganti saat membuat keputusan baru.

Anti-pattern paling berbahaya adalah suntingan diam-diam pada keputusan lama. Hal ini menghapus riwayat pembelajaran tim dan memicu asumsi keliru bagi engineer baru.

Never overwrite historical decisions; publish a new record to maintain lineage.

Reproduksi Mandiri di Mesin Lokal

Eksperimen dijalankan dengan Go standard library tanpa dependency luar.

```
cd labs/17-architecture-decision-record
go test -v ./...    # validasi parser & linter
go test -race ./... # cek race condition
go run ./cmd/demo   # jalankan simulasi 3 ADR
```

Lab ini zero-dependency dan dapat direproduksi langsung. Menjalankan demo dan test membuktikan parser dan linter bekerja secara mandiri di lokal.

Zero-dependency tests ensure fast, repeatable verification on any developer machine.

ADR Diubah Jadi Data yang Bisa Dicek

Memindahkan keputusan arsitektur dari teks bebas ke struktur yang terverifikasi.

PRINSIP REKAYASA

ADR terstruktur memungkinkan verifikasi otomatis relasi dokumen.
Linter menutup kelalaian mekanis yang sering luput dari review manusia.
Format standar membantu review; kualitas keputusan tetap ditentukan isi ADR.

Nilai utama lab ini adalah menjadikan keputusan arsitektur sebagai data yang bisa diuji. Linter menjaga integritas silsilah dokumen, sementara manusia fokus pada kualitas keputusan.

Parser makes ADR structured; linter enforces rules. Decisions remain human.

Parser membuat ADR terstruktur; linter baru memeriksa aturannya.

Parser makes ADR machine-readable; linter enforces structural invariants. Human wisdom remains irreplaceable.

Software Engineering Lab 17 – Architecture Decision Record

internal/adr/models.go • internal/adr/parser.go • internal/adr/linter.go
cmd/demo/main.go • tests/parser_test.go • tests/linter_test.go
README.md • content/02-master-draft.md

REPOSITORY

`github.com/lukman-ss/software-engineering-lab/tree/main/labs/17-architecture-decision-record`

Lukman / lukman-ss

Sumber: commit def0f5a
Test passing: 14 test cases, 0 failures.
Race detector: clean (go test -race ./...)
Simulasi in-memory string parsing; bukan proof production ADR governance completeness.